

Analysis Module Blueprint

This document is the implementation plan for the **Analysis** feature: one analysis per tab (or view), a **tabular** area for working data, and a **graphical** area for one to three highly interactive plots. It aligns with the existing project stack (`LoadedDataSet`, `ColumnSpec`, `ProjectDataManager`, `DataLoaderDialog` mapping patterns).

Related code (starting points):

- `src/rft_app/ui/widgets/analysis_widget.py` — layout shell (`tabular_frame`, `graphical_frame`, splitters).
 - `src/rft_app/project/models.py` — extend/fix `Analysis`, `Fluid`, `FluidType`; add analysis column metadata.
 - `src/rft_app/project/manager.py` — register/list/remove analyses; persistence hooks.
 - `src/rft_app/ui/main_window_KD.py` — tab creation passes `Analysis` + `ProjectDataManager` into `AnalysisWidget`.
-

Action Index

A single linear checklist of every action implied by this blueprint. Each item links back to the section that defines it in detail.

Phase A — Foundation: model, manager, conversion

- Completed:** Add `from __future__ import annotations` as the first line of `models.py`. → §2.1
- Completed:** Fix invalid annotations (e.g. `list[]` → `list[SomeType]`). → §2.1
- Completed:** Use `field(default_factory=...)` for all mutable defaults on dataclasses. → §2.1
- Completed:** Define `AnalysisColumnSpec` dataclass with fields `name`, `quantity_key`, `source_unit`, `display_unit`. → §2.2
- Completed:** Finalise `Analysis` fields: `name`, `source_dataset_name`, `row_indices`, `storage_data`, `display_data`, `column_specs`, `fluids`, `parameters`. → §2.3
- Completed:** Convert `Fluid` / `FluidType` placeholders to proper docstrings; ensure `FluidType` is defined before `Fluid` (or rely on postponed evaluation). → §2.4
- Open:** Add `src/rft_app/units/conversion.py` (or extend `units_manager.py`) with `to_display(value, quantity_key, source_unit, display_unit)` and batch helpers. → §2.5
- Open:** Document temperature conversion as affine (offset + scale), not a single factor. → §2.5
- Completed:** Add `self.analyses: list[Analysis] = []` to `ProjectDataManager`. → §3
- Open:** Implement `add_analysis`, `get_analysis`, `remove_analysis` and a unique-name helper. → §3
- Open:** Handle analyses that reference missing datasets on project load (warn / disable / clear ref). → §3
- Open:** Update `project/__init__.py` exports for the new types. → Phase A

Phase B — Build analysis data from project

- Implement `build_analysis_storage_from_source(project, analysis) -> None` (slice rows/columns, init `AnalysisColumnSpec`s). → Phase B

14. Implement `refresh_analysis_display_data(analysis, project) -> None` (convert `storage_data` → `display_data` per column). → [Phase B](#)
15. Call both from `AnalysisWidget` on load and after any unit combo change. → [Phase B](#)

Phase C — Analysis widget shell + tabular MVP

16. `AnalysisWidget` constructor: `(analysis, project, parent=None)`; store `self.analysis` and `self.project`. → [\\$4.2](#)
17. Split widget into `_build_ui()` and `_wire_signals()`; no business logic in the widget. → [\\$4.2](#)
18. Build `graphical_frame` (top) for 1–3 plots and `tabular_frame` (bottom) for table + toolbar. → [\\$4.1](#)
19. Replace bottom table with `QTableView` + custom `QAbstractTableModel` bound to `analysis.display_data`. → [\\$5.1](#)
20. Add a horizontal row of `QComboBox`es (one per column) above the `QTableView` for display units (Option A). → [\\$5.2](#)
21. Populate each combo from `STANDARD_QUANTITIES` / current `UnitSystem` (mirrors `DataLoaderDialog` mapping). → [\\$5.2](#)
22. On combo `currentIndexChanged`, update `AnalysisColumnSpec.display_unit`, rebuild `display_data` from `storage_data`, emit `dataChanged` / reset model. → [\\$5.2](#)
23. Enable sorting via `QSortFilterProxyModel`. → [\\$5.3](#)
24. Decide and document whether sort acts on display values only or reorders storage. → [\\$5.3](#)
25. Add a simple per-column text filter (`QLineEdit` per column or filter icon popup) using `setFilterKeyColumn` + `setFilterWildcard`. → [\\$5.4](#)

Phase D — Column add / remove

26. Build an "Add column" UI: pick a source column from `LoadedDataSet`, optional rename, append to `storage_data`, `display_data`, `column_specs`, combo row, then `insertColumns`. → [\\$5.5](#)
27. Build a "Remove column" action: warn/handle if plotted; drop from frames + specs; call `removeColumns`. → [\\$5.5](#)
28. Define rules for column subset alignment (v1: same row-subset only). → [\\$5.5](#)
29. Ensure all add/remove changes flow through `Analysis` (single source of truth). → [Phase D](#)

Phase E — First interactive plot

30. Add `pyqtgraph` to project dependencies. → [\\$8](#)
31. Spike one `pyqtgraph.PlotWidget` in `graphical_frame` with crosshair + right-click menu. → [\\$6.1](#)
32. Define `PlotSpec` (dataclass or dict inside `Analysis.parameters`) with `plot_index`, `x_column`, `y_column`, style, colour. → [\\$6.3](#)
33. Bind the spike plot to one series from `PlotSpec`; refresh on table `dataChanged` / structure change using `PlotDataItem.setData`. → [\\$6.3](#)
34. Add a basic right-click context menu (Reset view, Export, Toggle crosshair, series visibility). → [\\$6.4](#)

Phase F — Multi-plot and advanced interaction

35. Lay out 2–3 `PlotWidget` instances in a splitter / grid; visible count from settings. → [\\$6.2](#)
36. Crosshair + nearest-point readout via `SignalProxy` + `mouseMoved` + `TextItem`. → [\\$6.4](#)
37. Movable `InfiniteLine`s; connect `sigPositionChanged` to update `Analysis.parameters` (or refilter table). → [\\$6.4](#)
38. Legend with click-to-toggle series visibility. → [\\$6.4](#)

- 39. Add advanced filters in stages: numeric/range (Phase 2), Excel-style header dropdowns (Phase 3). → §5.4
- 40. (Optional) Brush selection on plot highlights matching rows in proxy model. → §6.4
- 41. Polish plots: image export, axis labels with units, consistent legend styling. → Phase F

Phase G — Persistence and main window wiring

- 42. Audit Analysis and nested types for pickle-safety: no QWidget / QColor ; use str for colours. → Phase G
- 43. MainWindowKD : tab creation builds Analysis , registers it with ProjectDataManager , passes it to AnalysisWidget . → Phase G
- 44. Define and document tab-close semantics (remove from project vs hide). → Phase G

Phase H — QA and UX

- 45. Large- DataFrame scroll/performance smoke test on the table and plots. → Phase H
- 46. Test empty project / no datasets / missing source dataset paths. → Phase H
- 47. Document keyboard shortcuts (if any) in user-facing notes. → Phase H

Cross-cutting decisions

- 48. Resolve open decisions before Phase C is hardened: row subset definition, sort/filter scope, add-column scope, default plot count. → §9
- 49. Track v1 completion against the success criteria checklist. → §10

1. Goals and non-goals

1.1 Goals

Area	Requirement
Data	Analysis uses a subset of project-loaded data; loaded datasets remain the source of truth.
Units	Per-column display units may differ from import units; combos drive conversion (reuse quantity → unit lists from units_manager).
Table	Bottom tabular region shows analysis data with Excel-like column behaviour: filter, sort, add/remove columns .
Plots	Top graphical region hosts 1–3 plots driven by table columns; high interactivity (context menu, move/drag series, identify points).
Architecture	Analysis is the model ; AnalysisWidget is the view; persistence via existing project pickle path.

1.2 Non-goals (initial phases)

- Floating/dockable analysis windows (deferred; can follow same model later).
- Full spreadsheet formula engine.
- Collaborative editing / undo stack across the whole app (optional later).

2. Data model (`project/models.py`)

Complete and stabilize types before large UI work.

2.1 File hygiene

1. Put `from __future__ import annotations` as the first line of `models.py` (before other imports).
2. Fix invalid annotations (e.g. `list[]` → `list[SomeType]`).
3. Use `field(default_factory=list)` (and similar) for mutable defaults on dataclasses.

2.2 `AnalysisColumnSpec` (new)

One row per logical column in the analysis table, aligned with `Analysis.display_data` columns.

Field	Purpose
<code>name</code>	Column header / key in <code>display_data</code> .
<code>quantity_key</code>	From source <code>ColumnSpec</code> (drives allowed units in combo).
<code>source_unit</code>	Unit in <code>LoadedDataSet</code> / storage slice (e.g. <code>m</code>).
<code>display_unit</code>	Unit shown in analysis (e.g. <code>ft</code>); updated when user changes combo.

Optional later: `source_column_name` if analysis column is renamed vs import.

2.3 `Analysis` (shape)

Field	Purpose
<code>name</code>	Unique analysis name in project.
<code>source_dataset_name</code>	Which <code>LoadedDataSet</code> the subset comes from (extend to multiple sources later if needed).
<code>row_indices</code>	<code>list[int]</code> or equivalent mask defining the subset (pickle-friendly).
<code>storage_data</code>	<code>pd.DataFrame \ None</code> — subset in source/import units (canonical for reconversion).

Field	Purpose
<code>display_data</code>	<code>pd.DataFrame \ None</code> — same shape/columns, values in display units for UI.
<code>column_specs</code>	<code>list[AnalysisColumnSpec]</code> — parallel to <code>display_data.columns</code> .
<code>fluids</code>	<code>list[Fluid]</code> (your existing types).
<code>parameters</code>	<code>dict[str, Any]</code> — bucket for evolving knobs without schema churn on every feature.

Rule: Mutations from the UI update `Analysis` fields, then refresh views. Never write converted numbers back into `LoadedDataSet`.

2.4 Fluid / FluidType

- Use proper **docstrings** (`"""..."""`).
- Ensure `FluidType` is defined before `Fluid` if not relying solely on postponed evaluation.

2.5 Unit conversion module

Add `src/rft_app/units/conversion.py` (or extend `units_manager.py`):

- API: `to_display(value, quantity_key, source_unit, display_unit) -> float` and batch column helpers.
- Document **temperature** as affine (offset + scale), not a single multiplicative factor.
- Stub linear ratios for other quantities first; refine per quantity as you add definitions.

3. Project manager (`project/manager.py`)

1. `self.analyses: list[Analysis] = []` (or dict by name; list + `_unique_name` matches datasets).
2. `add_analysis(analysis: Analysis) -> str`, `get_analysis(name) -> Analysis`, `remove_analysis(name)`.
3. `_unique_name`: reuse or generalise the pattern used for dataset names.
4. On **project load**, ensure analyses that reference missing datasets are handled (warn, disable tab, or clear `source_dataset_name` with user prompt — decide in Phase 1).

4. Analysis tab UI layout (`analysis_widget.py`)

4.1 Structure (keep current intent)

- **Top:** `graphical_frame` — container for 1–3 plot widgets in a splitter or horizontal layout.
- **Bottom:** `tabular_frame` — table + optional toolbar (filters, add column, reset view).

4.2 Constructor contract

```
AnalysisWidget(analysis: Analysis, project: ProjectDataManager, parent=None)
```

- Hold `self.analysis` and `self.project`.
- `_build_ui()` builds frames; `_wire_signals()` connects model changes to refresh.
- **No business logic** in the widget beyond orchestration — delegate to services (see §6).

5. Tabular region: table with Excel-like behaviour

5.1 Widget choice: `QTableView` + `QAbstractTableModel`

Recommendation: Use `QTableView` backed by a **custom** `QAbstractTableModel` (or a thin subclass of a pandas-oriented model), not `QTableWidget` alone.

Reason	Explanation
Sorting	<code>QTableView.setSortingEnabled(True)</code> works with models; for custom rules, proxy <code>QSortFilterProxyModel</code> .
Filtering	<code>QSortFilterProxyModel</code> gives per-column filter keys; advanced filters → custom proxy or delegate.
Performance	Large subsets stay in one <code>DataFrame</code> ; the model exposes rows without duplicating cell widgets per cell.
Unit row	First data row for unit combos is awkward in pure <code>QTableWidget</code> ; with a model you can use row 0 = units and rows <code>1..n</code> = data, or a dedicated header widget row above the view (cleaner).

Alternative: `QTableWidget` for prototyping only; plan migration to View/Model before filter/sort complexity grows.

5.2 Unit combo row

1. **Option A (preferred):** Horizontal row of `QComboBox` widgets in a `QWidget` **above** `QTableView`, one combo per column — aligns with headers without hacking row 0 of the model.
2. **Option B:** Row 0 inside the model marked "meta"; hide vertical header label for that row; sorting must skip row 0.

Steps:



1. Build combo list from `STANDARD_QUANTITIES` / current `UnitSystem` (same pattern as `DataLoaderDialog` mapping table).
2. On combo `currentIndexChanged`, update matching `AnalysisColumnSpec.display_unit`, rebuild `display_data` from `storage_data` via conversion module, emit `dataChanged` / reset model.

5.3 Sorting

1. Enable sorting on the **proxy** model (not always on the source model if row order is meaningful for “original import order”).
2. Decide: sort **display** values only (recommended) vs sort underlying storage (usually same if numeric).
3. If using a unit row inside the table, **exclude row 0** from sort or use Option A above.

5.4 Filtering (Excel-like)

Phase 1 — column text filter:

- `QLineEdit` per column in a filter bar, or filter icon in header opening a small popup.
- `QSortFilterProxyModel.setFilterKeyColumn` + `setFilterWildcard` / regex.

Phase 2 — range / numeric filters:

- Extend proxy or use a second-stage filter in Python that applies a mask to the `DataFrame` exposed by the model.

Phase 3 — filter UI:

- Header dropdown with checkboxes for distinct values (Excel-style) — more work; defer until Phase 1–2 stable.

5.5 Add / remove columns

Add column:

1. Dialog or sidebar: pick **source column** from `LoadedDataSet` (not already in analysis), optional rename.
2. Append column to `storage_data` and `display_data`, append `AnalysisColumnSpec`, append combo to unit row, `insertColumns` on model.

Remove column:

1. Confirm if column is **plotted** — warn or auto-remove from plot assignments.
2. Drop column from frames + specs, `removeColumns` on model.

Edge case: Adding a column that needs a **new** row subset — if all columns must share the same row index set, only allow columns from the same filtered index set; if not, define rules for ragged joins (usually avoid in v1).

6. Graphical region: 1–3 interactive plots

6.1 Library choice

Primary recommendation: **PyQtGraph**

- Native Qt integration, fast for large series, strong **interaction** (zoom, pan, crosshair).
- `PlotWidget`, `PlotDataItem`, `InfiniteLine` (movable lines), `ScatterPlotItem`.
- Context menus: override `getContextMenus` or install event filter on view box.

Alternative: `matplotlib` embedded in `FigureCanvasQTA`gg — familiar but heavier for drag/line tools; requires more custom code for Excel-like chart UX.

Decision step: Spike one `PlotWidget` with crosshair + right-click menu in `graphical_frame` before building three.

6.2 Layout

1. `QSplitter(Qt.Horizontal)` or `QGridLayout` with 1–3 `PlotWidget` instances.
2. User or analysis settings choose **visible count** (1–3); hide or remove extra widgets.

6.3 Data binding

1. `PlotSpec` (dataclass or dict inside `Analysis.parameters`): e.g. `plot_index`, `x_column`, `y_column`, line style, colour.
2. On **table model** `dataChanged` or **column structure changed**, re-read arrays from `display_data` (or `storage_data` + convert for axis labels only — be consistent).
3. Keep plot **series references** (`PlotDataItem`) to update `setData(x, y)` instead of full clear where possible.

6.4 Interactivity checklist (implement incrementally)

Feature	PyQtGraph direction
Right-click menu	<code>vb.menu</code> or <code>plotItem.ctrlMenu</code> ; add <code>QAction</code> s for "Reset view", "Export...", "Toggle crosshair", series visibility.
Pan / zoom	Default; optionally constrain axes.
Crosshair / point identify	<code>SignalProxy</code> + <code>mouseMoved</code> ; nearest index search on plotted arrays; <code>TextItem</code> or tooltip for <code>(x, y)</code> .
Grab/move lines	<code>InfiniteLine</code> with <code>movable=True</code> ; connect <code>sigPositionChanged</code> to update threshold in <code>Analysis.parameters</code> or refilter table (if that is the product intent).
Legend / series toggle	<code>plotItem.addLegend()</code> ; connect legend clicks to hide/show items.
Selection linked to table	Optional: brush selection on plot highlights rows in proxy model (advanced phase).

7. Phased implementation order (execute in sequence)

Phase A — Model and manager (foundation)

1. Fix `models.py` imports and syntax; add `AnalysisColumnSpec`.
2. Finalise `Analysis` fields (`source_dataset_name`, row subset, `storage_data`, `display_data`, `column_specs`, `fluids`, `parameters`).
3. Extend `ProjectDataManager` with `analyses` CRUD + unique names.
4. Add `units/conversion.py` with stubs + tests for at least one quantity (e.g. length `m` ↔ `ft`).
5. Update `project/__init__.py` exports if needed.

Phase B — Build analysis data from project

1. Service function `build_analysis_storage_from_source(project, analysis) -> None`: load `LoadedDataSet`, apply `row_indices`, copy selected columns into `storage_data`, initialise `AnalysisColumnSpec` (`source_unit` from `ColumnSpec`, `display_unit` default = source or project preference).
2. Function `refresh_analysis_display_data(analysis, project) -> None`: apply all column conversions from `storage_data` to `display_data`.
3. Call from `AnalysisWidget` on load and after unit combo changes.

Phase C — Tabular UI (MVP)

1. Replace bottom of `AnalysisWidget` with `QTableView` + `PandasTableModel` (or equivalent) bound to `analysis.display_data`.
2. Add unit combo row (Option A: widget row above table).
3. Enable sorting via `QSortFilterProxyModel`.
4. Add simple per-column filter (line edits or single filter row).

Phase D — Column add/remove

1. "Add column" UI + manager updates to `storage_data` / specs / recomputed `display_data`.
2. "Remove column" with plot dependency check.
3. Persist changes through `Analysis` only (table widget does not own truth).

Phase E — First plot

1. Embed one `pyqtgraph.PlotWidget` in `graphical_frame`.
2. Bind one series from `PlotSpec` in `analysis.parameters`.
3. Right-click menu with 2–3 actions (proof of concept).

Phase F — Multi-plot and advanced interaction

1. Layout for 2–3 plots; `PlotSpec` list per analysis.

2. Crosshair + nearest-point readout.
3. Movable `InfiniteLine` (or draggable curve if product requires — harder).
4. Polish: export image, axis labels with units, legend.

Phase G — Persistence and main window

1. Ensure `Analysis` + nested types are **pickle-safe** (no `QWidget`, no `QColor` on dataclasses — use `str` for colours).
2. `MainWindowKD`: creating a tab creates `Analysis`, registers with manager, passes to `AnalysisWidget`.
3. Closing tab: decide remove-from-project vs hide (document behaviour).

Phase H — QA and UX

1. Large `DataFrame` smoke test (scroll performance).
2. Empty project / no datasets / missing source dataset.
3. Document keyboard shortcuts (if any) in user-facing notes.

8. Dependencies

Add to project requirements when implementing plotting:

```
pyqtgraph
```

(Compatible with PyQt6; verify version pin in `requirements.txt` when introduced.)

9. Open decisions (resolve during Phase A–B)

1. **Row subset definition:** fixed `row_indices` vs reproducible filter spec (depth, well id, etc.).
 2. **Sort/filter scope:** proxy only (non-destructive) vs also write back to `display_data` (destructive).
 3. **Add column:** only from same `LoadedDataSet` as `source_dataset_name` for v1?
 4. **Plot count default:** always 3 placeholders vs lazy-create on demand.
-

10. Success criteria (module “done” for v1)

- ☐ New analysis tab shows table populated from a chosen dataset subset.
 - ☐ Unit combos per column convert correctly for at least length + one other quantity; temperature documented as TODO or implemented affine.
 - ☐ User can sort and filter without crashing; filter is clearly defined (even if simple).
 - ☐ User can add/remove columns; plots update or warn appropriately.
 - ☐ At least one interactive plot with context menu + point identification; at least one movable guide line **or** explicit deferral documented.
 - ☐ Save/load project restores analyses and reopens table + plots consistently.
-

Document version: 1.0 — blueprint for the Analysis module. Update phase checkboxes in this file or in `Open Tickets.md` as work completes.